

STEM4U — LEGO Computer Science & AI · Grades 6-8 (Engineers) · 30-Lesson Plan

Course Overview

Thirty weekly 60-minute sessions built on the LEGO® Education Computer Science & AI Kit (45522), coded with blocks on LEGO's Coding Canvas. Small groups of 4 students work in pairs (Driver/Navigator), single grade band (grades 6–8, the "Engineers"). Real tools. Real teams. Real engineers.

By the end, your student can independently design, build, and code a machine that senses its surroundings, decides what to do, stores and tracks data, reuses its own custom functions, and responds to an AI classifier they trained themselves — and can explain, in engineering terms, how and why it works.

This is a middle-school engineering course, not a toy. Students keep an engineering notebook, plan in pseudocode and flowcharts before they touch blocks, debug systematically instead of guessing, test against real cases, and defend design decisions out loud. We respect that these students are capable: vocabulary is real (algorithm, iteration, boolean, parameter, threshold, dataset, bias), builds are multi-step, and the extensions go deep.

AI runs through every unit. Students don't wait until Lesson 26 to meet artificial intelligence. Every lesson includes a 2-minute "AI Connection" that ties the day's coding idea to how real AI actually works — so by the time they train their own classifier in the final unit, it reads as the payoff of a year's worth of mental models, not a surprise. When we reach AI, we are honest about what it is: students train a **simple** image/pose/sound classifier on examples they collect, then use its output to drive a LEGO build. That is real machine learning at an honest scale — not "building their own ChatGPT."

Scope & sequence

Unit	Lessons	Big idea	What your student can do by unit's end
1 · Basics	1–5	A program is a precise algorithm a machine executes	Decompose a task, plan it in pseudocode/flowchart, then build and code it
2 · Loops	6–10	Computers iterate so we don't repeat ourselves	Use count, forever, and nested loops with a loop variable, and debug them
3 · Conditionals	11–15	Programs branch on logic	Code compound conditions and sensor thresholds that drive real decisions
4 · Variables	16–20	Programs store, update, and reason over data	Track multiple variables and use them to control behavior and keep state
5 · Functions	21–25	Good engineers package and reuse logic	Define functions (with parameters) and call them to organize a larger program
6 · AI & Data	26–30	Machines learn patterns from data we supply	Train a custom classifier, evaluate it, reason about bias, and drive a build with it

Standards & bridges: Lessons map to the CSTA K–12 CS Standards for grades 6–8 — primarily Algorithms & Programming (2-AP), with real connections to Data & Analysis (2-DA) and Impacts of Computing (2-IC). (See the **Standards Crosswalk** at the end for specific codes per unit.) The sequence deliberately builds the mental models — algorithms, iteration, boolean logic, variables, procedural abstraction, and data — that bridge directly to text languages like Python and to any block-to-text transition.

Note on unit order (6–8 vs. 3–5): This Engineers plan follows the LEGO 6–8 sequence, where **Functions** is the fifth unit. That differs from the younger 3–5 plan, which introduces **Events** as an early standalone unit. Here, event-handling is folded into Basics and Loops as a tool students

already command, freeing Unit 5 for genuine procedural abstraction — defining and calling functions, with parameters. This is the single biggest rigor step up from the 3–5 course.

Rotate the "Concept + demo" format so the same 10 minutes stays sharp week to week. Pick one each lesson: **(a) straight demo** (coach shows the idea), **(b) "coach ships a bug, team catches it"** (coach codes it wrong on purpose; pairs locate and name the defect), **(c) "trace it on paper first"** (predict the output or state by hand, then run to check), or **(d) "pair teaches pair"** (a pair who has it walks another pair through it). Debug lessons pair naturally with (b); review lessons with (d); conditionals and variables lessons with (c).

Engineering-notebook habit (every lesson). Each student keeps a notebook (paper or a shared doc). Minimum entry: the day's objective in their own words, their pseudocode or a quick flowchart, one bug they hit and how they isolated it, and one thing they'd change next time. The notebook is graded on honesty and reasoning, not neatness. It is also the running record for the design-challenge reflections and the AI dataset logs later in the year.

Every week gets a build/ownership hit. On mostly-code weeks, spend a spare minute on a **"reconfigure or make-it-yours" beat** — swap an attachment, re-gear a drivetrain, rename and brand the bot — so no week is screen-only and students keep owning the hardware.

The five mid-course design challenges are themed, each with a distinct share-out: **L5 Spec-build review** (each pair presents against the spec they were given) · **L10 Iteration showcase** ("show v1 vs. v2 and what changed") · **L15 Logic face-off** (two pairs solve the same decision problem, compare) · **L20 Systems demo day** (stateful builds run live) · **L25 Code-review gallery** (pairs read and praise one function in another pair's project). The course finale, **L30**, is a **capstone expo** with a short recorded demo for the parent note.

Progress layer: Each unit ends with a design-challenge lesson (5, 10, 15, 20, 25, 30). At each, the coach sends home a **short parent note** ("here's what your student mastered this unit, in plain language") plus a **photo of the design-challenge build**, so families see concrete growth across the year.

How to read a lesson (for coaches with no CS background)

Every lesson follows the same shape and the same clock, so you can teach it cold: an "I can..." goal, the CS concept and what it builds on, a 2-line AI Connection to read aloud, a materials note (most weeks the model is already built from earlier in the unit), a **timed 60-minute flow**, Driver/Navigator pairing, differentiation (Support / Extension / Reteach), an exit check, and an optional at-home idea. Extensions double as **fast-finisher** tasks; the Reteach line is your quick fix when a pair stalls. You are a facilitator, not the answer key — when a pair is stuck, ask "what did you expect, what happened, and where do they differ?" before you touch their screen.

The 60-minute clock (every lesson): Warm-up **8** · Concept + demo **10** · Build/Code **30** · Challenge **7** · Share + pack-up **5**.

Model reuse: Each unit's LEGO model is built once, in the unit's first lesson, and stays assembled for the rest of the unit. Later lessons in a unit are mostly coding (with small, deliberate reconfigurations), so class time goes to engineering thinking, not rebuilding.

Pairing: Driver runs the device/blocks; Navigator reads the plan, checks against pseudocode, and leads debugging. They **swap at the halfway mark** of Build/Code every lesson — non-negotiable, because both students must be able to drive and to reason. Groups of 4 run as two pairs sharing one kit; if a group is odd, make one **rotating trio** (Driver, Navigator, Tester) that rotates at each third.

Lesson 1 — Basics: Setup, the Canvas, and Your First Program

Objective (I can...): I can pair the hub, navigate the Coding Canvas, and build and run a short program, and I can explain what makes a program an algorithm. **CS concept:** A program is a precise, ordered algorithm a machine executes exactly as written · **Builds on:** No prior lesson — course launch.

AI Connection (2 min): Face unlock on a phone is a program *and* an AI model working together — the program runs a fixed procedure, the AI makes a judgment call. *First-principles question:* "If you had to make a computer 'recognize a cat' from scratch, what exact, checkable steps could it follow?" (Collect ideas — we'll return to this all year.)

Materials: LEGO CS & AI kit 45522 (1 per pair), a device per pair running Coding Canvas, a printed kit-inventory card per pair, engineering notebooks. **No full build today** — use the kit's quick-start model (hub + one motor) so the focus is the toolchain. The unit model is built next lesson.

60-minute flow - Warm-up (8): "Literal robot" — a student gives the coach precise spoken steps to draw a shape on the board; the coach follows them exactly, flaws and all. Point: a machine executes your algorithm literally, so precision is the engineer's job, not the machine's. - **Concept + demo (10):** Power and pair the hub over Bluetooth. Tour the Canvas: block palette by category, the workspace, Run/Stop, and where live sensor values appear. Build a 3-block program (start → run motor → stop) and run it on the quick-start model. Define *algorithm* on the board. - **Build/Code (30):** Each pair (1) pairs their hub, (2) completes a guided tour scavenger hunt — locate five block categories and the port view, (3) builds and runs a 3-block program on the quick-start model. Then they modify it: change the motor's speed and duration and predict the effect before running. *Swap Driver/Navigator at 15 min.* Coach circulates to confirm every hub is paired and every pair can Stop. - **Challenge (7):** Reorder the blocks to change the behavior, predict the new result in the notebook, then run to check the prediction. - **Share + pack-up (5):** Each pair runs their program and states one thing that surprised them. Cables coiled, hub off, parts in tray.

Pair roles: Driver pairs the hub and drags blocks; Navigator runs the scavenger hunt and records in the notebook. Swap at 15 min. (*Trio: add a Tester who runs each version and reports.*)

Differentiation: *Support* — a printed palette map keyed by icon. *Extension / fast-finisher* — add a sound and a light so three outputs fire in sequence. *Reteach* — re-pair together slowly; have the student name each control before using it.

Exit check: Pair has a paired hub, ran and modified a 3-block program, and can define "algorithm" in their own words. **At-home (optional):** Write a precise 8-step algorithm for making a sandwich; mark the one step a literal robot would most likely get wrong. ~10 min.

Lesson 2 — Basics: Build the Model & Sequencing

Objective (I can...): I can build the unit model and order instructions so the machine performs a multi-step sequence reliably. **CS concept:** Sequencing and execution order (statements run top to bottom; order changes outcome) · **Builds on:** Lesson 1 (running a program).

AI Connection (2 min): A voice assistant runs an ordered pipeline: capture audio → transcribe → interpret → respond. *First-principles question:* "Which two of those steps absolutely cannot be swapped, and why?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **This is the unit's build lesson** — build the Basics model now and keep it assembled through Lesson 5.

60-minute flow - Warm-up (8): Sequence sabotage — pairs get a 6-step process written out of order and reorder it, then identify which single swap would be most destructive. Connect to execution order. - **Concept + demo (10):** Run the same block set in two different orders; show the different outcomes. Introduce reading a program top-to-bottom as "tracing" and model tracing one line at a time out loud. - **Build/Code (30):** Pairs **build the Basics model** per the Canvas instructions (the one build for the unit), then write and run a 5-step sequence (e.g., light on → drive forward → turn → play tone → stop), tracing each step before running. *Swap at 15 min.* - **Challenge (7):** Reorder the same blocks into a deliberately different, sensible behavior; record both the old and new trace in the notebook. - **Share + pack-up (5):** Each pair shows original vs. reordered program and names what the order changed. **Leave the model built** on its tray.

Pair roles: During the build, Driver connects bricks while Navigator reads and checks steps; they swap for coding. Swap at 15 min. (*Trio rotates build/read/test.*)

Differentiation: *Support* — a worked 5-step example to adapt, not copy. *Extension / fast-finisher* — write an 8-step sequence with two outputs running close together. *Reteach* — trace the program aloud, one line at a time, running after each line.

Exit check: Pair produces a working 5-step sequence and can trace it, explaining why order matters. **At-home (optional):** Write the exact start-up sequence for a device you use (game console, microwave); note what breaks if two steps swap. ~10 min.

Lesson 3 — Basics: Decomposition & Flowcharts

Objective (I can...): I can break a complex task into sub-problems and represent the plan as a flowchart before I code. **CS concept:** Decomposition and algorithm representation (flowcharts) · **Builds on:** Lesson 2 (sequencing).

AI Connection (2 min): A vision model decomposes "is this a cat?" into layered sub-checks — edges, then shapes, then parts, then the whole. *First-principles question:* "What's the advantage of solving many small sub-problems instead of one big one?"

Materials: LEGO CS & AI kit + device per pair, flowchart symbol reference card (start/end, process, decision, arrow), notebooks. **Model reuse:** Basics model from Lesson 2 — code plus one small build tweak.

60-minute flow - Warm-up (8): "Decompose it" — pairs take a big task ("run a lemonade stand") and break it into three levels of sub-tasks on a whiteboard. Point: decomposition is how engineers make big problems tractable. - **Concept + demo (10):** Introduce flowchart symbols. Coach picks a behavior (patrol forward, then signal), decomposes it aloud, and draws a flowchart — then maps each box to a block. (*Rotate format: "trace it on paper first" — read a flowchart and predict the behavior.*) - **Build/Code (30):** Pairs choose a two-part behavior, decompose it into sub-tasks, draw a flowchart in the notebook, then code it on the built model so blocks match the flowchart. **Make-it-yours beat:** add one attachment and name the build. *Swap at 15 min.* - **Challenge (7):** Add a third sub-task to the flowchart and the code, keeping chart and code in sync. - **Share + pack-up (5):** Pairs show flowchart beside the running program and point out the one-to-one match. Model stays built.

Pair roles: Navigator owns the flowchart; Driver codes from it. *Swap at 15 min. (Trio: third verifies each flowchart box has a matching block.)*

Differentiation: *Support* — a half-drawn flowchart to finish. *Extension / fast-finisher* — decompose a three-part behavior with a sub-task that repeats. *Reteach* — draw one box, code it, confirm, then move to the next box.

Exit check: Pair's flowchart and working program correspond box-for-block. **At-home (optional):** Draw a flowchart for your morning routine with at least six process boxes. ~10 min.

Lesson 4 — Basics: Pseudocode, Abstraction & Iterative Design

Objective (I can...): I can write pseudocode, use abstraction to hide detail, and improve my program across two test-and-revise cycles. **CS concept:** Pseudocode and abstraction; the iterative design loop (plan → build → test → refine) · **Builds on:** Lesson 3 (decomposition and flowcharts).

AI Connection (2 min): Engineers spec an AI feature in plain language first — "IF a face is recognized with high confidence, THEN unlock" — then implement and refine it over many iterations. *First-principles question:* "Why is the first version of almost anything engineered never the last?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, notebooks. **Model reuse:** Basics model from Lesson 2 — code plus one small tweak.

60-minute flow - Warm-up (8): "Spec swap" — each pair writes 4–5 lines of pseudocode for a behavior, then trades with another pair; your received spec is your mission. Ambiguous specs get built wrong on purpose — that's the lesson in precision. - **Concept + demo (10):** Coach writes pseudocode, maps it to blocks, runs it, finds it imperfect, and revises — narrating the iterative loop. Name *abstraction*: the plan hides motor ports and timings so you can reason about the big steps. (*Rotate format:* "coach ships a bug, team catches it.") - **Build/Code (30):** Pairs build the received spec: refine pseudocode, code it, run it, then run **at least one deliberate revision** and log in the notebook what changed and why. **Make-it-yours beat:** adapt one brick to the mission theme. *Swap at 15 min.* - **Challenge (7):** Change one line of the spec (e.g., add a condition in words) and update the code to match. - **Share + pack-up (5):** Pairs read the pseudocode aloud, run to prove the match, and state their one revision. Model stays built.

Pair roles: Navigator owns the plan sheet and the revision log; Driver translates and runs. Swap at 15 min. (*Trio: third logs the revision rationale.*)

Differentiation: *Support* — sentence-framed pseudocode to complete. *Extension / fast-finisher* — write pseudocode that includes a repeated step and a decision in words. *Reteach* — code one line together, then let them continue; do the revision together.

Exit check: Pair's pseudocode maps one-to-one to working blocks, and they can name one revision and the reason for it. **At-home (optional):** Write pseudocode for a daily task, then write a v2 that fixes one weakness. ~10 min.

Lesson 5 — Basics: Design Challenge — Spec-Build Review

Objective (I can...): I can take a written spec, plan it, build and code it, and present my solution against the requirements. **CS concept:** Applying sequencing, decomposition, flowcharts, and pseudocode to a specified problem · **Builds on:** Lessons 1–4. *Also serves as flex/catch-up — if earlier lessons ran long, use today to finish and consolidate.*

AI Connection (2 min): Shipping an AI feature means meeting a spec and defending your choices in a review — the same loop you'll run today. *First-principles question:* "If two teams meet the same spec differently, how do we judge which design is better?"

Materials: LEGO CS & AI kit + device per pair, a printed spec card per pair (a short requirements list, e.g., "≥4 ordered actions, uses two different outputs, has a clear start and end"), plan sheets, notebooks. **Model already built** (Basics model) — the 30 minutes is plan + code + test, small reconfigs only.

60-minute flow - Warm-up (8): Vocabulary carousel — fast whiteboard prompts: algorithm, sequence, decomposition, flowchart, pseudocode, abstraction, iteration. - **Concept + demo (10):** Hand out the spec. Coach models decomposing the spec into requirements, drafts pseudocode for one, then steps back. Set the expectation: you present *against the spec*. - **Build/Code (30):** Pairs decompose the spec, plan it (flowchart or pseudocode), and build/code a solution that meets every requirement on the built model. Coach coaches with questions, doesn't solve. *Swap at 15 min.* - **Challenge (7):** Add one requirement of your own and update the plan so it still documents the whole program. - **Share + pack-up (5): Spec-build review** — each pair presents their build, pointing to where it satisfies each requirement, and takes one question. **Coach photographs each build for the unit parent note.**

Pair roles: Co-plan; then Driver codes / Navigator checks against the spec; swap at 15 min. (*Trio rotates plan/code/review.*)

Differentiation: *Support* — a simpler spec (3 requirements) with a partial plan. *Extension / fast-finisher* — a spec requiring a repeated action and a decision described in words. *Reteach* — hand them their Lesson 4 pseudocode as a starting scaffold.

Exit check: Pair delivers a program meeting every spec requirement and can point to where each is satisfied.

Progress layer: Send home the **Unit 1 parent note** ("your student can decompose a problem, plan it, and build it to spec") with the **build photo**. **At-home (optional):** Write a one-paragraph spec for a gadget, then list how you'd test that it meets the spec. ~10 min.

Lesson 6 — Loops: Build the Model & Count Loops

Objective (I can...): I can build the loops model and use a count-controlled loop to replace repeated blocks.

CS concept: Iteration; count-controlled loops; a loop is abstraction (one "repeat N" stands in for N copies) ·

Builds on: Lesson 5 (sequences we now repeat efficiently).

AI Connection (2 min): Training an AI model is iteration at scale — the same update step runs over data thousands of times. *First-principles question:* "Why would repeating a tiny improvement many times produce a big result?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **This is the unit's build lesson** — build the Loops model now and keep it assembled through Lesson 10.

60-minute flow - Warm-up (8): "Compress it" — coach writes out a long repeated instruction list; pairs rewrite it as the shortest possible instruction. Reveal: "repeat N times" is a count-controlled loop, and it's a form of abstraction. - **Concept + demo (10):** Show four copied drive blocks, then replace them with one block inside a repeat-4 loop; run both and confirm identical behavior. Point out how much less code can go wrong. - **Build/Code (30):** Pairs **build the Loops model** (the unit's one build), then program a count loop (drive → pause, repeated N times). They record in the notebook what one copied-out equivalent would look like. *Swap at 15 min.* - **Challenge (7):** Change the count and predict the total run time before testing; log prediction vs. actual. - **Share + pack-up (5):** Pairs show the loop, state the count, and explain why the loop version is easier to maintain. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator counts iterations aloud and logs the prediction. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a palette card highlighting loop blocks. *Extension / fast-finisher* — put two actions inside the loop body. *Reteach* — count iterations out loud together on the first run.

Exit check: Pair replaces repeated blocks with a working count loop and can explain the abstraction. **At-home (optional):** Find a repetitive task and write it as "repeat ___ times { ___ }." ~10 min.

Lesson 7 — Loops: Loop Bodies & Systematic Debugging

Objective (I can...): I can put several actions in one loop body and debug a loop using a trace. **CS concept:** Multi-statement loop bodies; debugging discipline (reproduce → isolate → fix → retest) · **Builds on:** Lesson 6 (single-action count loop).

AI Connection (2 min): Each training round repeats the same multi-step update on new data. *First-principles question:* "If one step inside that repeated round is wrong, how many times does the mistake get made?"

Materials: LEGO CS & AI kit + device per pair, "broken program" cards, notebooks. **Model reuse:** Loops model from Lesson 6 — code only.

60-minute flow - Warm-up (8): "Find the defect" — coach shows a short block program on the board with one visible bug; pairs locate it and state what it would do wrong before any fix. Introduce the four-step debugging routine. - **Concept + demo (10):** Build a loop body with three actions (drive → tone → wait); run it, point out the whole body repeats as a unit. Then plant a bug (one block outside the loop) and model the debugging routine aloud. - **Build/Code (30):** Pairs code a 3-action repeating pattern inside one loop. **Debug task (5 min):** each pair receives a broken program (a block sits outside the loop, or the count is off); they reproduce the symptom, isolate the line, fix it, and retest, logging each step in the notebook. *Swap at 15 min.* - **Challenge (7):** Add a fourth action to the loop body and re-verify the pattern still holds together. - **Share + pack-up (5):** Pairs demo the pattern and report how they isolated their bug. Model stays built.

Pair roles: Driver codes the loop body, Navigator leads the debugging routine and logs it. *Swap at 15 min. (Trio: third owns reproducing the symptom.)*

Differentiation: *Support* — a 2-action body with a worked example. *Extension / fast-finisher* — a 5-action body plus a planted bug of their own for another pair. *Reteach* — build the body outside a loop, confirm it works, then drag it inside.

Exit check: Pair runs a loop with a 3+ action body and can describe the four-step routine they used to fix the bug. **At-home (optional):** Describe a real bug you've seen (a game glitch, a jammed printer) and how you'd isolate its cause. ~10 min.

Lesson 8 — Loops: Forever Loops & Controlled Stopping

Objective (I can...): I can use a forever loop and stop it in a controlled way with an event or a condition. **CS concept:** Infinite loops and controlled termination · **Builds on:** Lesson 7 (loop bodies).

AI Connection (2 min): An always-on assistant runs a forever loop, sampling the microphone until a wake word fires. *Impacts prompt:* "An always-listening device is convenient — what's the tradeoff for the people living with it?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **Model reuse:** Loops model from Lesson 6 — code only.

60-minute flow - Warm-up (8): "What stops it?" — coach starts an endless chant; pairs propose a clean stopping rule (a signal, not just "give up"). Introduce controlled termination. - **Concept + demo (10):** Demo a forever loop (blinking light), stop it with the Stop button, then better, with a button event — contrast an abrupt kill with a designed exit. - **Build/Code (30):** Pairs program a forever loop (constant patrol or blink) on the built model, then engineer a controlled stop via an event or a sensor condition. They note in the notebook the difference between the two stopping methods. *Swap at 15 min.* - **Challenge (7):** Use one event to start the forever loop and a different event to stop it, so the model has a clear on/off life cycle. - **Share + pack-up (5):** Pairs demo start and controlled stop and explain why a designed exit beats the Stop button. Model stays built.

Pair roles: Driver codes the loop, Navigator owns the stop design. Swap at 15 min. (*Trio rotates start/loop/stop.*)

Differentiation: *Support* — stop via the Stop button, then add one event stop with help. *Extension / fast-finisher* — add a condition that also halts the loop on a sensor threshold. *Reteach* — run the forever loop, feel that it never ends, then add the stop together.

Exit check: Pair runs a forever loop and halts it through a designed event or condition, not just Stop. **At-home (optional):** Name a system that "runs until told to stop" (a thermostat, a server); describe its stop condition. ~10 min.

Lesson 9 — Loops: Nested Loops & the Loop Variable

Objective (I can...): I can nest loops and use the loop counter to make patterns that change each pass. **CS concept:** Nested iteration; using the loop index/variable · **Builds on:** Lesson 8 (loops and controlled stopping).

AI Connection (2 min): Training nests loops: for each epoch (outer), loop through every example (inner).

First-principles question: "If it's 5 epochs over 100 examples, how many single updates run in total — and why does nesting make that easy to express?"

Materials: LEGO CS & AI kit + device per pair, grid paper, "broken program" cards, notebooks. **Model reuse:** Loops model from Lesson 6 — code only.

60-minute flow - Warm-up (8): "Rows and columns" — pairs compute total actions for several outer×inner combinations and spot the multiplication pattern. Connect to nested loops. - **Concept + demo (10):** Build an inner loop (tone 3×) inside an outer loop (repeat 4×); run it so students hear 4 sets of 3. Then show using the loop counter so each outer pass drives a little farther. (*Rotate format:* "trace it on paper first" — *predict total and order.*) - **Build/Code (30):** Pairs code a nested loop where the inner body repeats and the outer pass changes something using the loop index (e.g., drive distance grows each pass). **Debug task (5 min):** receive a program with inner/outer counts swapped; find why the total is wrong and fix it, logging the diagnosis. *Swap at 15 min.* - **Challenge (7):** Change the inner or outer count and predict the new total before running. - **Share + pack-up (5):** Pairs demo, state the total inner runs, and explain how the loop variable changed each pass. Model stays built.

Pair roles: Driver builds the inner loop, Navigator adds the outer and tracks the index. Swap at 15 min. (*Trio: third leads the debug.*)

Differentiation: *Support* — a worked nested example without the loop variable. *Extension / fast-finisher* — use the index in two places (distance and tone pitch). *Reteach* — build and test the inner loop alone first, then wrap the outer around it.

Exit check: Pair runs a working nested loop and can explain inner vs. outer counts and how the index was used. **At-home (optional):** Write a nested loop in pseudocode for a seating chart (rows of seats); state the total seats. ~10 min.

Lesson 10 — Loops: Design Challenge — Iteration Showcase

Objective (I can...): I can design a machine that uses loops, then iterate it into a measurably better v2. **CS concept:** Applying loops (count, forever, nested) with iterative refinement and testing · **Builds on:** Lessons 6–9. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): AI improves by iterating — version after version, each tested. *First-principles question:* "What would you measure to prove your v2 is actually better than your v1, not just different?"

Materials: LEGO CS & AI kit + device per pair (with camera), plan sheets, notebooks. **Model already built** (Loops model) — the build time is plan + code + test, small reconfigs only.

60-minute flow (*As the flex lesson, today's clock trims Build/Code to ~22 to make room for an 8-min AI preview. If a pair needs the full build time, run the preview as a whole-group demo.*) - **Warm-up (8):** Loop sort — pairs classify example programs as count, forever, or nested, and name a good use for each. (*Theme: Iteration showcase.*) - **Concept + demo (10):** Challenge: build a looped machine (patrol, light show, drummer) that must use at least one loop, then produce a v2 that improves one measurable thing (smoother, faster, more reliable). Coach models choosing a metric. (*Rotate format: "pair teaches pair."*) - **Build/Code (~22):** Pairs plan and code v1 with a loop, pick a metric, then build v2 and record both measurements in the notebook. *Swap at the halfway mark.* - **Meet a real classifier — hands-on preview (8):** Pairs open **Google Teachable Machine** in the browser and show cards, objects, or hand gestures to the camera using a pre-made or demo model, watching the confidence bars move. **No training yet** — just see a classifier react live and notice it's never 100% sure. *This previews Unit 6, where they train their own.* **Privacy note, said aloud:** Teachable Machine trains in the browser, on-device; no login is needed and images aren't stored unless someone chooses to save them. - **Share + pack-up (5):** Showcase — pairs present v1 vs. v2 with their metric and point out the loop; each names one thing the classifier got right or wrong. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator owns the metric and measurements; swap at 15 min. (*Trio rotates plan/code/measure.*)

Differentiation: *Support* — a single-loop target with a provided metric. *Extension / fast-finisher* — require a nested loop and two measured improvements. *Reteach* — reuse a Lesson 7–9 loop as the v1 starting point.

Exit check: Pair delivers a looped build and can show, with a measurement, how v2 improved on v1.

Progress layer: Send home the **Unit 2 parent note** ("your student can use loops and improve a design with evidence") with the **build photo**. **At-home (optional):** Pick something you do repeatedly; design a v2 of how you do it and name the metric that would show it's better. ~10 min.

Lesson 11 — Conditionals: Build the Model & If / Then

Objective (I can...): I can build the conditionals model and use an if/then so the machine decides whether to act. **CS concept:** Selection; boolean conditions in an if/then · **Builds on:** Lesson 10 (loops set the stage for decisions inside them).

AI Connection (2 min): A classifier's output drives a decision: "IF confidence that it's a dog > 80%, THEN label it dog." *First-principles question:* "What should the program do when confidence is right at the borderline?"

Materials: LEGO CS & AI kit + device per pair, trigger objects / color cards, notebooks. **This is the unit's build lesson** — build the Conditionals model now and keep it assembled through Lesson 15.

60-minute flow - Warm-up (8): "True statement, now act" — coach reads conditions; pairs state whether each is true right now and what action should follow. Introduce *boolean*: a condition is either true or false. - **Concept + demo (10):** Demo an if/then: if the distance sensor reads under a value, then stop; otherwise nothing. Show the live sensor reading so the condition's truth is visible. - **Build/Code (30):** Pairs **build the Conditionals model** (the unit's one build), then program one if/then driven by a sensor (object near → act). They write the condition in words in the notebook. *Swap at 15 min.* - **Challenge (7):** Change the condition's threshold and verify the action fires only when the condition is true. - **Share + pack-up (5):** Pairs demo the true case and the "nothing happens" case and read their condition aloud. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator reads the condition and watches the live value. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a palette card for the if/then block and comparison operators. *Extension / fast-finisher* — make the "then" a two-step action. *Reteach* — move the object while narrating "true... false..." against the live reading.

Exit check: Pair's model acts only when its boolean condition is true. **At-home (optional):** Write three if/then rules that govern a device in your home; state the condition precisely. ~10 min.

Lesson 12 — Conditionals: If / Then / Else & Tracing Logic

Objective (I can...): I can use if/then/else for two-way branching and trace which branch runs for a given input. **CS concept:** Two-way selection; tracing boolean logic · **Builds on:** Lesson 11 (if/then).

AI Connection (2 min): Face unlock branches: IF recognized, THEN unlock; ELSE stay locked and log the attempt. *Impacts prompt:* "Fast unlock helps most people — who could it lock out unfairly, and why?"

Materials: LEGO CS & AI kit + device per pair, trigger objects / color cards, notebooks. **Model reuse:** Conditionals model from Lesson 11 — code only.

60-minute flow - Warm-up (8): "Which branch?" — coach reads an if/then/else and an input value; pairs call the branch and the result. Model tracing on the board. - **Concept + demo (10):** Demo if/then/else: object near → green light + forward; else → red light + wait. Run both branches and trace each. (*Rotate format: "trace it on paper first."*)

- **Build/Code (30):** Pairs code an if/then/else with two clearly distinct outcomes from one condition, then trace, in the notebook, which branch fires for each of two test inputs. *Swap at 15 min.* -

Challenge (7): Make each branch a two-step action (light + motion) and re-trace. - **Share + pack-up (5):** Pairs demo both branches and state which condition selected which. Model stays built.

Pair roles: Driver codes the "then," Navigator designs and traces the "else"; swap at 15 min so each owns a branch. (*Trio: third runs both test inputs.*)

Differentiation: *Support* — the "then" provided; students add the "else" from framed pseudocode.

Extension / fast-finisher — nest an if inside one branch. *Reteach* — trigger each branch by hand, label it, then code it.

Exit check: Pair's model shows two distinct outcomes correctly selected by one condition, and they can trace which branch runs for a given input. **At-home (optional):** Write an if/then/else for a real decision and trace it for two different situations. ~10 min.

Lesson 13 — Conditionals: Compound Conditions (AND / OR / NOT)

Objective (I can...): I can combine conditions with AND, OR, and NOT so my program reasons over more than one fact. **CS concept:** Compound boolean conditions; logical operators · **Builds on:** Lesson 12 (if/then/else).

AI Connection (2 min): A security system might act only if motion AND an unrecognized face are both detected. *First-principles question:* "When would requiring AND make a system safer, and when would it make it miss something important?"

Materials: LEGO CS & AI kit + device per pair, two kinds of triggers (object + color), a truth-table handout, notebooks. **Model reuse:** Conditionals model from Lesson 11 — code only.

60-minute flow - Warm-up (8): "True/false math" — pairs fill a small truth table for AND and OR using everyday statements. Introduce NOT as flipping true/false. - **Concept + demo (10):** Demo a compound condition: if (object near) AND (red seen) then act; change AND to OR and show how the behavior widens. Show NOT on a single condition. - **Build/Code (30):** Pairs code one AND condition and one OR condition driving different actions, then predict and test the result for each combination of inputs, logging a mini truth table in the notebook. *Swap at 15 min.* - **Challenge (7):** Add a NOT so an action fires only when a condition is *false*; verify it. - **Share + pack-up (5):** Pairs demo an AND vs. an OR case and explain the difference in plain English. Model stays built.

Pair roles: Driver codes, Navigator owns the truth table and calls each test case. *Swap at 15 min. (Trio: third runs the combinations.)*

Differentiation: *Support* — provide the truth table pre-filled; students match it to one compound condition. *Extension / fast-finisher* — build a three-condition expression and predict its table. *Reteach* — test each simple condition alone, then combine two and watch the result change.

Exit check: Pair's program uses at least one compound condition correctly and they can explain AND vs. OR.

At-home (optional): Write a real rule that needs AND and one that needs OR (e.g., "wear a coat if cold AND going outside"). ~10 min.

Lesson 14 — Conditionals: Sensor Thresholds & Multi-Way Decisions (+ Debug)

Objective (I can...): I can use sensor thresholds to build a multi-way decision (more than two outcomes). **CS concept:** Thresholds; chained/else-if selection · **Builds on:** Lesson 13 (compound conditions).

AI Connection (2 min): A model sets a confidence threshold — "act only above 70%" — and can have tiers: very sure, somewhat sure, unsure. *First-principles question:* "What goes wrong if the threshold is set too low? Too high?"

Materials: LEGO CS & AI kit + device per pair, objects at varied distances / color set, "broken program" cards, notebooks. **Model reuse:** Conditionals model from Lesson 11 — code only.

60-minute flow - Warm-up (8): "Pick the cutoffs" — pairs set two distance cutoffs that split "close / medium / far" and defend them. Introduce multi-way decisions. - **Concept + demo (10):** Demo chained conditions: if very close → back up; else if medium → slow; else → cruise. Show the live value crossing each threshold. (*Rotate format: "coach ships a bug, team catches it" — a threshold set backwards.*) - **Build/Code (30):** Pairs code a three-outcome decision from sensor thresholds. **Debug task (5 min):** receive a program whose thresholds overlap or are mis-ordered so one branch is unreachable; diagnose and fix, logging the reasoning. *Swap at 15 min.* - **Challenge (7):** Tune the cutoffs and test how the behavior boundaries shift. - **Share + pack-up (5):** Pairs demo all three outcomes at three distances and explain their cutoffs. Model stays built.

Pair roles: Driver codes, Navigator moves the object and calls each threshold crossing. *Swap at 15 min. (Trio: third leads the debug.)*

Differentiation: *Support* — provide the two cutoff values. *Extension / fast-finisher* — four outcomes, or combine a threshold with a compound condition. *Reteach* — watch the live number and set one cutoff at a time, testing each.

Exit check: Pair's model reliably produces three distinct behaviors across sensor thresholds, and they fixed the unreachable-branch bug. **At-home (optional):** Describe a real "smart" behavior with three tiers (e.g., screen brightness by light level) as chained conditions. ~10 min.

Lesson 15 — Conditionals: Design Challenge — Logic Face-Off

Objective (I can...): I can design an autonomous machine that makes layered decisions, and compare my logic with another team's. **CS concept:** Applying selection, compound conditions, and thresholds to an autonomous build · **Builds on:** Lessons 11–14. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): A self-driving system layers decisions every moment — obstacle? pedestrian? light color? *First-principles question:* "What sensor + rule would make YOUR machine make a smart, defensible decision?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, trigger objects, notebooks. **Model already built** (Conditionals model) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow - Warm-up (8): Logic review — pairs match scenarios to if/then, if/then/else, compound, and multi-way. (*Theme: Logic face-off.*) - **Concept + demo (10):** Challenge: build an autonomous "guard," "sorter," or "navigator" that decides among actions using at least a compound condition or a threshold. Coach models a flowchart for the decision, then hands off. - **Build/Code (30):** Pairs plan (flowchart) and code a layered decision on the built model. *Swap at 15 min.* - **Challenge (7):** Add a second decision or wrap the decision in a loop so it decides continuously. - **Share + pack-up (5): Logic face-off** — pair two teams that met the *same* goal; each traces their logic for the same test input, then together they judge which design is clearer and why (often more than one correct answer). **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks the logic and flowchart; swap at 15 min. (*Trio rotates plan/code/trace.*)

Differentiation: *Support* — a single-condition target with a partial flowchart. *Extension / fast-finisher* — require a compound condition inside a loop. *Reteach* — reuse a Lesson 13–14 program as a template.

Exit check: Pair delivers an autonomous model that decides correctly with layered logic and can trace it against another team's solution. **Progress layer:** Send home the **Unit 3 parent note** ("your student can program a machine to make its own layered decisions") with the **build photo**. **At-home (optional):** Design a "smart" rule-set for a gadget using AND/OR and a threshold; write it as a flowchart. ~10 min.

Lesson 16 — Variables: Build the Model & Storing Data

Objective (I can...): I can build the variables model, create a clearly named variable, store a value, and read it. **CS concept:** Variables as named storage; naming and data values · **Builds on:** Lesson 15 (decisions we can now base on stored data).

AI Connection (2 min): A model stores everything it "knows" as numbers in memory — weights, counts, scores. *First-principles question:* "What's the difference between a program that *remembers* across runs and one that starts fresh every time?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **This is the unit's build lesson** — build the Variables model now and keep it assembled through Lesson 20.

60-minute flow - Warm-up (8): "Name it well" — coach shows poorly named vs. clearly named variables ("x" vs. "scoreRed"); pairs rank them and state a naming rule. Good names are documentation. - **Concept + demo (10):** Create a variable, set it to a starting value, display it on screen, then change it and watch the display update. Name the difference between a value and the box that holds it. - **Build/Code (30):** Pairs **build the Variables model** (the unit's one build), then create a clearly named variable, set a starting value, and display it. They record the variable's name and purpose in the notebook. *Swap at 15 min.* - **Challenge (7):** Set a different starting value and re-run; predict the displayed value first. - **Share + pack-up (5):** Pairs show their named variable and its value and justify the name. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator names the variable and reads its value. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a worked example with a well-named variable to adapt. *Extension / fast-finisher* — create two variables of different purposes. *Reteach* — write the value on paper, then mirror it on screen so "stored value" is concrete.

Exit check: Pair creates a clearly named variable, stores and displays a value, and can justify the name. **At-home (optional):** Name five variables a video game would need to track and say what each holds. ~10 min.

Lesson 17 — Variables: Updating & Accumulating Data

Objective (I can...): I can update a variable (increment, accumulate) to track events over time. **CS concept:** Updating variables; accumulation patterns · **Builds on:** Lesson 16 (creating/storing a variable).

AI Connection (2 min): A model keeps running tallies — examples seen, correct predictions, running error. *First-principles question:* "Why does tracking a running total matter more than a single snapshot when something is learning?"

Materials: LEGO CS & AI kit + device per pair, trigger object/button, notebooks. **Model reuse:** Variables model from Lesson 16 — code only.

60-minute flow - Warm-up (8): "Running total" — pairs track two different counts by hand as the coach fires events quickly; connect to "change variable by N." - **Concept + demo (10):** Demo "change score by 1" inside an event so each trigger adds one; show the running total. Then demo accumulating by a sensor reading (add the distance each loop). - **Build/Code (30):** Pairs program a counter (each event adds 1) and a separate accumulator (sum a value over a loop), both displayed. They log the update pattern in the notebook. *Swap at 15 min.* - **Challenge (7):** Add a reset event that sets the counter back to 0 without touching the accumulator. - **Share + pack-up (5):** Pairs demo counting, accumulating, and resetting, and explain the difference between the two. Model stays built.

Pair roles: Driver codes the updates, Navigator fires events and reads the totals. Swap at 15 min. (*Trio rotates code/trigger/read.*)

Differentiation: *Support* — provide the event; students add the "change by 1." *Extension / fast-finisher* — count by a step other than 1, or track two independent counters. *Reteach* — fire one event, watch 0→1, then let them predict the next.

Exit check: Pair's counter and accumulator both update correctly, and the counter resets independently. **At-home (optional):** Track something for a day (steps, pages); write it as a variable that gets updated each time. ~10 min.

Lesson 18 — Variables: Variables Driving Decisions & State

Objective (I can...): I can use a variable inside a condition to make my program change its behavior based on state. **CS concept:** Variables combined with conditionals; the idea of program *state* · **Builds on:** Lesson 17 (updating) and Lesson 12 (if/then/else).

AI Connection (2 min): A model acts on stored data: IF accuracy on test examples $\geq$ target, THEN "trained enough." *First-principles question:* "What number would honestly tell you a system is 'good enough' to use?"

Materials: LEGO CS & AI kit + device per pair, trigger object/button, notebooks. **Model reuse:** Variables model from Lesson 16 — code only.

60-minute flow - Warm-up (8): "State changes everything" — coach describes a machine behaving differently at count 0 vs. count 5; pairs name the "state" that caused the change. Introduce *state*. - **Concept + demo (10):** Demo: count events; if the score reaches a target, then switch behavior (celebration mode); else keep counting. Point out the variable is the program's memory of its state. - **Build/Code (30):** Pairs code a counter plus an if/then that changes behavior when the variable hits a chosen value, and the behavior should visibly persist (stay in the new state). They note in the notebook what "state" their program is in before and after. *Swap at 15 min.* - **Challenge (7):** Add an else branch for "not yet," and a second threshold for a third state. - **Share + pack-up (5):** Pairs demo crossing the threshold and the state change, and explain what the variable "remembers." Model stays built.

Pair roles: Driver codes the if/then, Navigator owns the counter and narrates the state. *Swap at 15 min. (Trio rotates.)*

Differentiation: *Support* — provide the counter; students add the condition from framed pseudocode.

Extension / fast-finisher — three states from two thresholds. *Reteach* — set the target to 2 so it triggers fast, then raise it.

Exit check: Pair's model changes and holds a new behavior when the variable reaches the target, and they can describe the state. **At-home (optional):** Describe a device with clear "states" (phone: locked / unlocked / low-battery); name what variable tracks each. ~10 min.

Lesson 19 — Variables: Multiple Variables & Data You Can Reason About

Objective (I can...): I can use several variables together and reason about relationships between them. **CS concept:** Multiple variables; simple data relationships (comparisons, differences) · **Builds on:** Lesson 18 (a variable in a decision).

AI Connection (2 min): A model never relies on one number — it juggles accuracy, speed, examples per category, error. *First-principles question:* "Why can a single metric lie? Give an example where one number looks great but the system is bad."

Materials: LEGO CS & AI kit + device per pair, trigger objects/buttons, notebooks. **Model reuse:** Variables model from Lesson 16 — code only.

60-minute flow - Warm-up (8): "Two numbers, one judgment" — pairs decide a winner from two stats (points AND fouls) and defend it. Introduce reasoning over multiple variables. - **Concept + demo (10):** Demo two variables (score and a countdown timer), updating each, displaying both, and a condition that compares them (end when score > timer). - **Build/Code (30):** Pairs program at least two variables — a score counting up on events and a timer/count changing over a loop — then add a condition that compares or combines them (win/lose, lead by 3). They log both variables' meaning. *Swap at 15 min.* - **Challenge (7):** Compute and display a derived value (e.g., difference between the two) and react to it. - **Share + pack-up (5):** Pairs demo both variables changing and explain the relationship their program reasons about. Model stays built.

Pair roles: Each partner owns one variable; swap at 15 min to review the other's and co-write the comparison. (*Trio: third owns the derived value.*)

Differentiation: *Support* — one variable pre-built; students add the second. *Extension / fast-finisher* — three variables with a combined end condition. *Reteach* — build and test one variable fully, then copy the pattern for the second.

Exit check: Pair runs a program with two+ correctly labeled variables and a working condition that reasons about their relationship. **At-home (optional):** Pick a sport or game; list three stats it tracks and one judgment that needs two of them together. ~10 min.

Lesson 20 — Variables: Design Challenge — Systems Demo Day

Objective (I can...): I can design a stateful system that stores, updates, and reasons over data to drive behavior. **CS concept:** Applying variables with events, loops, and conditionals to a stateful build · **Builds on:** Lessons 16–19. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): An adaptive game AI tracks your performance and adjusts difficulty — data drives its choices. *Impacts prompt:* "Adaptive systems can help some players and frustrate others — what's a fair way to design that?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, trigger objects, notebooks. **Model already built** (Variables model) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow - Warm-up (8): Variables review — create, update, use-in-decision, multiple, state, derived value. (*Theme: Systems demo day.*) - **Concept + demo (10):** Challenge: build a stateful system (scorekeeper, timer game, resource meter) that uses at least one variable plus an event, a loop, and a conditional, and clearly changes state. Coach models the plan, then hands off. - **Build/Code (30):** Pairs plan and code the stateful system on the built model, documenting each variable's role in the notebook. *Swap at 15 min.* - **Challenge (7):** Add a second variable, a reset, or a win/lose condition that compares two variables. - **Share + pack-up (5): Systems demo day** — pairs run their system live while another pair tries to make it change state; creators explain the variables behind the behavior. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks the variable logic; swap at 15 min. (*Trio rotates plan/code/test.*)

Differentiation: *Support* — a counter-and-threshold target with a partial plan. *Extension / fast-finisher* — require two variables, a derived value, and a win condition. *Reteach* — reuse a Lesson 18–19 program as a template.

Exit check: Pair delivers a working stateful system that stores, updates, and uses variables to drive behavior, with each variable documented. **Progress layer:** Send home the **Unit 4 parent note** ("your student can build a system that tracks and reasons over data") with the **build photo**. **At-home (optional):** Design a simple game on paper; list every variable it needs and when each changes. ~10 min.

Lesson 21 — Functions: Build the Model & Define Your First Function

Objective (I can...): I can build the functions model, define a named function, and call it to run a block of logic. **CS concept:** Procedural abstraction — defining and calling a function (procedure) · **Builds on:** Lesson 20 (programs are now big enough to need organizing).

AI Connection (2 min): Engineers package AI behavior into reusable functions — "detectFace()," "classify()" — and call them wherever needed instead of rewriting them. *First-principles question:* "If you had to run the same 10-step routine in five places, what's the risk of copying it five times?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **This is the unit's build lesson** — build the Functions model now and keep it assembled through Lesson 25.

60-minute flow - Warm-up (8): "Name the routine" — pairs take a repeated real-world routine (a free-throw routine, a pre-flight check) and give it a single name, then describe it by that name. That naming *is* abstraction — the core of a function. - **Concept + demo (10):** Show a program with the same 4-block routine pasted three times, then define it once as a function (e.g., "beepAndBlink") and call it three times; run both to show identical behavior with far less code. Name *define* vs. *call*. - **Build/Code (30):** Pairs **build the Functions model** (the unit's one build), then define one function that does a small routine and call it at least twice in their program. They log the function's name and job in the notebook. *Swap at 15 min.* - **Challenge (7):** Call the function from inside a loop and predict how many times the routine runs. - **Share + pack-up (5):** Pairs show the definition and the calls and explain why one definition beats three copies. **Leave the model built.**

Pair roles: Driver defines/codes, Navigator names the function and counts the calls. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a palette card and a worked define/call example to adapt. *Extension / fast-finisher* — define two functions. *Reteach* — write the routine once, confirm it works, then wrap it in a function definition together.

Exit check: Pair defines a named function and calls it two+ times, and can explain define vs. call. **At-home (optional):** Name three routines in your day; write each as a function name and list its steps once. ~10 min.

Lesson 22 — Functions: Parameters — One Function, Many Behaviors

Objective (I can...): I can give a function a parameter so one definition handles many cases. **CS concept:** Parameters and arguments · **Builds on:** Lesson 21 (define/call).

AI Connection (2 min): A reusable "classify(image)" takes the image as a parameter — same function, different input each time. *First-principles question:* "Why is a function that takes an input more powerful than one hard-wired to a single value?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): "Same recipe, different amount" — coach gives a routine that only differs by a number ("drive 10," "drive 25"); pairs spot that the one thing that changes should be the parameter. -

Concept + demo (10): Define a function with one parameter (e.g., "driveFor(distance)" or "flash(times)"); call it with different arguments and show different behavior from one definition. (*Rotate format:* "trace it on paper first" — predict each call's result.) - **Build/Code (30):** Pairs convert a fixed-value function from last lesson into a parameterized one, then call it with at least three different arguments. They log, in the notebook, what the parameter controls. *Swap at 15 min.* - **Challenge (7):** Add a second parameter so one call controls two things (distance and speed). - **Share + pack-up (5):** Pairs demo three calls with different arguments and explain what the parameter changed. Model stays built.

Pair roles: Driver codes the parameter, Navigator chooses and logs the test arguments. *Swap at 15 min.* (*Trio: third runs each argument.*)

Differentiation: *Support* — provide the parameterized definition; students supply varied arguments.

Extension / fast-finisher — two parameters, or a parameter used inside a loop count. *Reteach* — change the hard-coded number by hand a few times, then replace it with a parameter together.

Exit check: Pair's parameterized function produces different, correct behavior for different arguments. **At-home (optional):** Write a real routine as a function with a parameter (e.g., "cook(minutes)"); show three calls. ~10 min.

Lesson 23 — Functions: Decomposing a Big Program into Functions

Objective (I can...): I can break a larger program into several well-named functions that work together. **CS concept:** Decomposition via functions; modular design · **Builds on:** Lesson 22 (parameters) and Lesson 3 (decomposition).

AI Connection (2 min): A real AI pipeline is many functions chained — capture, preprocess, classify, respond — each testable on its own. *First-principles question:* "Why is a program made of small named functions easier to fix than one giant block?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): "Carve it up" — pairs take a described multi-stage behavior and split it into 3–4 named functions on paper, deciding what each owns. - **Concept + demo (10):** Demo a main program that calls three functions in order (e.g., "startup()", "patrol(distance)", "shutdown()"); show the main reads like an outline. Emphasize each function is tested alone before assembly. - **Build/Code (30):** Pairs plan a behavior as 3 functions, build and test each function independently, then assemble a short main that calls them. They log each function's single job in the notebook. *Swap at 15 min.* - **Challenge (7):** Reuse one function in two different places in the main to prove its generality. - **Share + pack-up (5):** Pairs read their main program as an outline of function calls and explain how modularity made testing easier. Model stays built.

Pair roles: Each partner owns and tests one or two functions, then they assemble the main together; swap at 15 min. (*Trio: three functions, one each, assemble together.*)

Differentiation: *Support* — provide two function stubs to fill; students write the third and the main.

Extension / fast-finisher — four functions, one parameterized. *Reteach* — build and test one function fully, then add the next, assembling incrementally.

Exit check: Pair's program is organized into 3+ working, well-named functions called from a readable main.

At-home (optional): Outline a project (cleaning your room) as a main that calls 4 named functions; define one. ~10 min.

Lesson 24 — Functions: Reuse, Documentation & Refactoring (+ Debug)

Objective (I can...): I can refactor repeated code into functions, document them, and debug a function-based program. **CS concept:** Refactoring; code documentation; debugging modular code · **Builds on:** Lesson 23 (modular design).

AI Connection (2 min): Teams reuse and document shared AI functions so others can build on them safely and give credit. *First-principles question:* "If a bug lives inside a function called in five places, how many places do you fix — and why is that the point?"

Materials: LEGO CS & AI kit + device per pair, a "messy program" card (works but has duplicated code and a planted bug), notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): "Spot the duplicate" — coach shows a program with the same chunk pasted twice; pairs mark the duplication that should become a function. Introduce *refactoring*: improving structure without changing behavior. - **Concept + demo (10):** Take a working-but-messy program, refactor the duplicated chunk into one function, re-run to prove behavior is unchanged, and add a one-line comment/label documenting the function. (*Rotate format:* "coach ships a bug, team catches it.") - **Build/Code (30):** Pairs receive the messy program. They (1) refactor duplicated blocks into a function, (2) document each function with a short note of its job, (3) find and fix the single planted bug — fixing it once inside the function. They log the refactor and the fix in the notebook. *Swap at 15 min.* - **Challenge (7):** Parameterize the refactored function so it also replaces a near-duplicate that differed by one value. - **Share + pack-up (5):** Pairs show before/after structure and report the bug and where one fix solved it everywhere. Model stays built.

Pair roles: Driver refactors/codes, Navigator documents and leads the debug. *Swap at 15 min. (Trio: third owns documentation.)*

Differentiation: *Support* — point to the duplicated chunk to extract. *Extension / fast-finisher* — refactor two separate duplications. *Reteach* — extract the function together, re-run to prove sameness, then fix the bug.

Exit check: Pair refactors duplicated code into a documented function, behavior unchanged, and fixes the bug in one place. **At-home (optional):** Find repeated instructions in a recipe or manual; rewrite them as one named, documented step. ~10 min.

Lesson 25 — Functions: Design Challenge — Code-Review Gallery

Objective (I can...): I can design a well-structured program built from functions and read another team's code to give useful feedback. **CS concept:** Applying functions, parameters, and modular design; peer code review · **Builds on:** Lessons 21–24. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): Professional AI code is reviewed by peers before it ships — readability and reuse matter as much as "it runs." *First-principles question:* "What would make another engineer trust and reuse your function without rewriting it?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, a short code-review checklist (clear names? one job each? reused? documented?), notebooks. **Model already built** (Functions model) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow - Warm-up (8): Functions review — define/call, parameters, decomposition, refactoring, documentation. (*Theme: Code-review gallery.*) - **Concept + demo (10):** Challenge: build a program that does something interesting using at least two functions, one with a parameter and one reused. Coach models the review checklist on a sample. (*Rotate format: "pair teaches pair."*) - **Build/Code (30):** Pairs plan and code a modular program meeting the function requirements on the built model, documenting each function. *Swap at 15 min.* - **Challenge (7):** Reuse one function from inside a loop or a second function (composition). - **Share + pack-up (5): Code-review gallery** — pairs rotate to another team's device, read one function using the checklist, and write one specific piece of praise and one question in that team's notebook. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks structure against the checklist; swap at 15 min. (*Trio rotates plan/code/review.*)

Differentiation: *Support* — one function with a parameter and a provided main outline. *Extension / fast-finisher* — three functions with composition and a parameter passed through. *Reteach* — reuse a Lesson 23 program and add one parameter.

Exit check: Pair delivers a modular program meeting the function requirements and gives one useful, specific review comment to another team. **Progress layer:** Send home the **Unit 5 parent note** ("your student can organize a real program into reusable functions and review code") with the **build photo**. **At-home (optional):** Review a sibling's or friend's plan for anything; give one specific praise and one question, the way we did in class. ~10 min.

Lesson 26 — AI & Data: How Machines Learn from Data

Objective (I can...): I can explain how a classifier learns from examples, collect a small dataset, and run a first training. **CS concept:** Supervised learning basics; a dataset as labeled examples · **Builds on:** Lesson 25 (we'll wrap AI calls in functions and conditions). *Privacy: training runs in the browser on-device via Google Teachable Machine — no login, and images aren't stored unless someone saves them.*

AI Connection (2 min): This is the real thing: you supply labeled examples, and the model learns the pattern that separates them — the honest version of every "AI learns from examples" note this year. *First-principles question:* "How would you prove to a skeptic that the model actually learned, and didn't just memorize your face?"

Materials: Device per pair with camera and browser (Google **Teachable Machine**), LEGO CS & AI kit, notebooks set up as a **dataset log** (category → number of examples → notes). **This is the AI unit's build lesson** — build the AI responder model now and keep it assembled through Lesson 29.

60-minute flow - Warm-up (8): "Be the classifier" — coach shows mixed picture cards; pairs sort into two categories and write the rule they used. Point: a classifier learns a rule like that, but from many examples, not a sentence. - **Concept + demo (10):** Walk through the honest pipeline on Teachable Machine: create two classes, add examples, train, test. Say clearly what it is (a simple image classifier) and what it is NOT (not building a chatbot or "its own AI"). Restate the privacy facts aloud. - **Build/Code (30):** Pairs **build the AI responder model** (the unit's one build), then create a 2-category image classifier in Teachable Machine, capture several examples each, train, and test it live. They start the **dataset log** in the notebook (examples per category). *Swap at 15 min so both capture and operate.* - **Challenge (7):** Add a third category; note whether accuracy on the first two changed. - **Share + pack-up (5):** Pairs show their classifier labeling live and read their dataset log. **Leave the model built.** Close the Teachable Machine tab (nothing saved).

Pair roles: Driver captures and labels examples, Navigator runs tests and keeps the dataset log. Swap at 15 min. (*Trio rotates capture/train/test.*)

Differentiation: *Support* — two very distinct categories (bright card vs. dark card). *Extension / fast-finisher* — add examples from varied angles and lighting. *Reteach* — capture examples together and watch the confidence bars rise as examples are added.

Exit check: Pair trains a working 2-category classifier and can explain, honestly, what it learned and what it is not. **At-home (optional):** List three everyday AI classifiers (spam filter, photo tagging, voice wake-word) and what examples each likely learned from. ~10 min.

Lesson 27 — AI & Data: Connecting a Trained Model to a Build

Objective (I can...): I can use my trained classifier's output as the input to a conditional that drives my LEGO build. **CS concept:** Model output as program input; classification feeding selection · **Builds on:** Lesson 26 (training) and the conditionals unit. *Privacy: on-device in the browser, no login, images not saved.*

AI Connection (2 min): The classifier's label and confidence become the condition in an IF/THEN — the same selection logic from Unit 3, now driven by AI. *First-principles question:* "Should the build still act if the model is only 55% sure? Who decides the cutoff?"

Materials: Device per pair (Teachable Machine + Coding Canvas), LEGO CS & AI kit, category cards/objects, notebooks. **Model reuse:** AI responder model from Lesson 26 — mostly code.

60-minute flow - Warm-up (8): "Label then act" — pairs decide, for three labels, what the build should do for each, and a confidence cutoff below which it does nothing. - **Concept + demo (10):** Demo reading the classifier's label and confidence, then an if/then: if label is "A" and confidence above threshold, then the build does X; else it waits. Wrap the reaction in a function from Unit 5. - **Build/Code (30):** Pairs connect their trained model so a recognized category drives the model (classify → if label/confidence → function that acts). They log the confidence threshold they chose and why. *Swap at 15 min.* - **Challenge (7):** Add a second category that calls a different reaction function. - **Share + pack-up (5):** Pairs demo the classifier driving the build and explain their confidence cutoff. Model stays built.

Pair roles: Driver codes the if/then and functions, Navigator presents items to the camera and reads label/confidence. Swap at 15 min. *(Trio: third logs thresholds.)*

Differentiation: *Support* — one label-to-action link provided; students add the second. *Extension / fast-finisher* — three labels, each a different function, with a confidence gate. *Reteach* — narrate "label = A, confidence high → true → act" while testing.

Exit check: Pair's build reacts correctly and only above a stated confidence threshold based on the classifier's output. **At-home (optional):** Describe a product that acts on an AI's guess; what should it do when the AI is unsure? ~10 min.

Lesson 28 — AI & Data: Evaluating & Improving a Model (Data Literacy)

Objective (I can...): I can test my model's accuracy, find where it fails, and improve it by changing the dataset. **CS concept:** Model evaluation; refining a model with better/more balanced data · **Builds on:** Lesson 27 (using a model). *Privacy: on-device in the browser, no login, images not saved.*

AI Connection (2 min): This is why "more and better examples help": a model is only as good as its dataset. *First-principles question:* "If it keeps confusing two categories, is the fix more code — or better data? How would you tell?"

Materials: Device per pair (Teachable Machine), LEGO CS & AI kit, varied test items, notebooks (dataset log continues). **Model reuse:** AI responder model from Lesson 26 — retrain and code.

60-minute flow - Warm-up (8): "Fooled you" — coach shows a classifier (or describes one) that fails on an edge case; pairs hypothesize what was missing from its training data. - **Concept + demo (10):** Demo a weak classifier, run a quick accuracy check on held-back test items, identify the failing category, add better/more varied examples, retrain, and show accuracy improve. Introduce *balanced dataset*. - **Build/Code (30):** Pairs run their own accuracy check (e.g., 10 test trials per category, tally correct in the dataset log), diagnose the weakest category, add targeted examples (varied angle, lighting, background), retrain, and re-test. They record before/after accuracy. *Swap at 15 min.* - **Challenge (7):** Introduce a tricky look-alike category and improve the dataset until the model handles it. - **Share + discussion (5):** Pairs report accuracy change, then discuss **why** the model still errs — was the dataset unbalanced, trained in one lighting, or on only one person? Name one better example that would help. Model stays built.

Pair roles: Driver captures new examples, Navigator runs the accuracy trials and records the tally. Swap at 15 min. (*Trio: third owns the test protocol.*)

Differentiation: *Support* — improve one category and measure just that. *Extension / fast-finisher* — balance three categories and report a per-category accuracy. *Reteach* — add examples one at a time, re-testing so each improvement is visible.

Exit check: Pair shows measured improvement after changing the dataset and can name one data-quality reason the model erred. **At-home (optional):** Explain to a family member, with your own example, why an AI trained on a narrow dataset can be unreliable. ~10 min.

Lesson 29 — AI & Data: Bias, Fairness & Responsible Use

Objective (I can...): I can identify how biased data causes unfair AI, test for it in my own model, and argue for responsible use. **CS concept:** Dataset bias and its impacts; privacy and responsible use · **Builds on:** Lesson 28 (data quality). *Privacy: on-device in the browser, no login, images not saved — and this lesson makes that a design value, not just a fact.*

AI Connection (2 min): Real AI systems have made unfair decisions because their training data didn't represent everyone — this is a top concern for working engineers. *First-principles question:* "If your classifier only ever saw examples that look like you, who might it fail for, and whose responsibility is that?"

Materials: Device per pair (Teachable Machine), LEGO CS & AI kit, notebooks, a short "bias scenario" card set. **Model reuse:** AI responder model from Lesson 26 — test and retrain.

60-minute flow - Warm-up (8): "Who's missing?" — pairs read a short scenario (a classifier trained on only one kind of example) and predict who it will fail for and the real-world harm. - **Concept + demo (10):** Demo (or set up) a deliberately biased model — e.g., trained in only bright light or on only one object orientation — and show it fail on the under-represented case. Name *bias* as a data problem with people-impacts, and connect to the privacy choices (on-device, nothing saved) as responsible design. - **Build/Code (30):** Pairs deliberately build a biased version of their classifier (narrow examples), document its failure on a held-out case, then fix it by broadening the dataset and re-testing. They write a short notebook entry: the bias, who it would affect, and the fix. *Swap at 15 min.* - **Challenge (7):** Draft two "responsible-use rules" for a product built on their model (e.g., confidence gate, human override, what data it should and shouldn't keep). - **Share + discussion (5):** Pairs present their biased-vs-fixed result and read one responsible-use rule. Discuss the privacy tradeoff: convenience vs. keeping data private. Model stays built.

Pair roles: Driver builds the biased/fixed datasets, Navigator documents the impact and the rules. Swap at 15 min. (*Trio: third runs the held-out test.*)

Differentiation: *Support* — provide the bias scenario and one rule stem to complete. *Extension / fast-finisher* — analyze a second, subtler bias and propose a test for it. *Reteach* — contrast two trainings side by side (narrow vs. broad) so the effect is concrete.

Exit check: Pair demonstrates a biased model, fixes it with better data, and can state who biased AI harms and one responsible-use rule. **At-home (optional):** Find or recall a news example of unfair or privacy-invasive AI; write what data choice likely caused it. ~10 min.

Lesson 30 — AI & Data: Capstone Expo (Design Challenge)

Objective (I can...): I can design and present a complete machine that senses, decides, tracks data, reuses functions, and responds to an AI model I trained. **CS concept:** Integrating classifiers with events, conditionals, variables, and functions · **Builds on:** Lessons 1-29 — the whole course. *Privacy: on-device in the browser, no login, images not saved. Also serves as flex/catch-up — use today to finish and polish anything outstanding from the AI unit.*

AI Connection (2 min): Today's build senses, decides, counts, reuses your own functions, and responds to an AI YOU trained and evaluated — every unit of the year working as one system. *First-principles question:* "Which idea from this year is doing the most work in your capstone, and why?"

Materials: Device per pair (Teachable Machine + Coding Canvas), LEGO CS & AI kit, plan sheets, props, notebooks. **Model already built** — reuse the AI responder model (L26) or a small reconfig; the 30 min is plan + code + test + polish, not build-from-scratch.

60-minute flow - Warm-up (8): Capstone pitch — pairs state in one sentence what their machine does and which trained category drives it. (*Theme: Capstone expo.*) - **Concept + demo (10):** Challenge: build an interactive project where a trained classifier drives a LEGO build that also uses at least one conditional and at least one variable or function. Coach models connecting a recognition to a behavior and restates the privacy points. - **Build/Code (30):** Pairs plan and code the integrated capstone on the already-built model — classifier output → condition → a function that acts, with a variable tracking something meaningful. *Swap at 15 min.* -

Challenge (7): Add a branch so a different recognition leads to a different outcome, or a count that changes the behavior over time. - **Share + pack-up (5): Capstone expo** — each pair demos their machine live while the coach records a short video clip for the parent note; audience names one concept they see working.

Photograph each build. Close Teachable Machine tabs (nothing saved).

Pair roles: Co-plan; Driver codes / Navigator directs the demo and cues recognitions; swap at 15 min. (*Trio rotates plan/code/present.*)

Differentiation: *Support* — one recognition, one conditional, one function; partial plan. *Extension / fast-finisher* — multiple recognitions, branching outcomes, a variable, and a parameterized function. *Reteach* — reuse their Lesson 27-28 trained model and add one new behavior.

Exit check: Pair delivers a working machine that uses a trained, evaluated classifier to drive behavior, integrating a conditional and a variable or function, and can explain how it works. **Progress layer:** Send home the **Unit 6 (course finale) parent note** ("your student can train and evaluate an AI model and build a machine that senses, decides, tracks data, and reuses its own functions") with the **build photo** and the short demo clip — the year's capstone. **At-home (optional):** Write a half-page "engineer's summary" of your capstone: what it does, how the AI drives it, and one thing you'd improve with more time. ~10 min.

Standards Crosswalk

Aligned to the **CSTA K-12 CS Standards, grades 6-8** — primarily **Algorithms & Programming (2-AP)**, with real connections to **Data & Analysis (2-DA)** and **Impacts of Computing (2-IC)**. This is a targeted map, not a claim of full coverage. *Maps to the Virginia CS Standards of Learning, grades 6-8 (confirm exact SOL numbers).*

Unit (Lessons)	Skills taught	CSTA grades 6-8 codes targeted
1 • Basics (1-5)	Algorithms, sequencing, decomposition, flowcharts, pseudocode, abstraction, iterative design, building to spec	2-AP-13 (decompose problems into parts); 2-AP-10 (use flowcharts/pseudocode to design algorithms); 2-AP-17 (test and refine); 2-AP-15 (seek and use feedback — at the spec review)
2 • Loops (6-10)	Count, forever, and nested loops; loop variable; controlled termination; systematic debugging; measured iteration	2-AP-12 (combine control structures incl. nested loops); 2-AP-17 (systematically test and refine); 2-AP-19 (document programs)
3 • Conditionals (11-15)	If/then, if/then/else, compound conditions (AND/OR/NOT), sensor thresholds, multi-way decisions, tracing logic	2-AP-12 (compound conditionals and control structures); 2-AP-17 (test with a range of cases); 2-AP-13 (decompose a decision problem)
4 • Variables (16-20)	Create/name variables, update/accumulate, variables in decisions, program state, multiple variables and relationships	2-AP-11 (clearly named variables, operations on values); 2-AP-12 (variables with control structures); 2-AP-19 (document variables' roles)
5 • Functions (21-25)	Define/call functions, parameters and arguments, modular decomposition, reuse, refactoring, documentation, code review	2-AP-14 (create procedures with parameters for reuse); 2-AP-13 (decompose into subproblems); 2-AP-16 (incorporate/reuse code with attribution); 2-AP-19 (documentation); 2-AP-15 (peer feedback/review)
6 • AI & Data (26-30)	Train a classifier on collected data; model output as input; evaluate accuracy; refine with balanced data; bias, fairness, privacy, responsible use	2-DA-08 (collect data with computational tools and transform it to be reliable); 2-DA-09 (refine computational models based on their data); 2-IC-21 (bias and accessibility in technology design); 2-IC-23 (public-vs-private tradeoffs)
Pair programming (every lesson)	Driver/Navigator roles, swapping, distributed tasks, collaboration	2-AP-18 (distribute tasks and maintain a timeline when collaborating); 2-AP-15 (seek and incorporate feedback)

Cross-cutting: the debugging tasks (L7, L9, L14, L24) and test protocols reinforce **2-AP-17** and **2-AP-19**; the engineering-notebook habit and function documentation reinforce **2-AP-19**; the design-challenge reviews and the L25 code-review gallery reinforce **2-AP-15** and **2-AP-18**; the AI-impacts prompts (L8, L12, L20) and the L28-L29 data-literacy, bias, and privacy work reinforce **2-DA-08/09**, **2-IC-21**, and **2-IC-23**; the honest-AI framing throughout (a simple classifier trained on-device, never "their own ChatGPT") is maintained from the first AI Connection to the capstone.